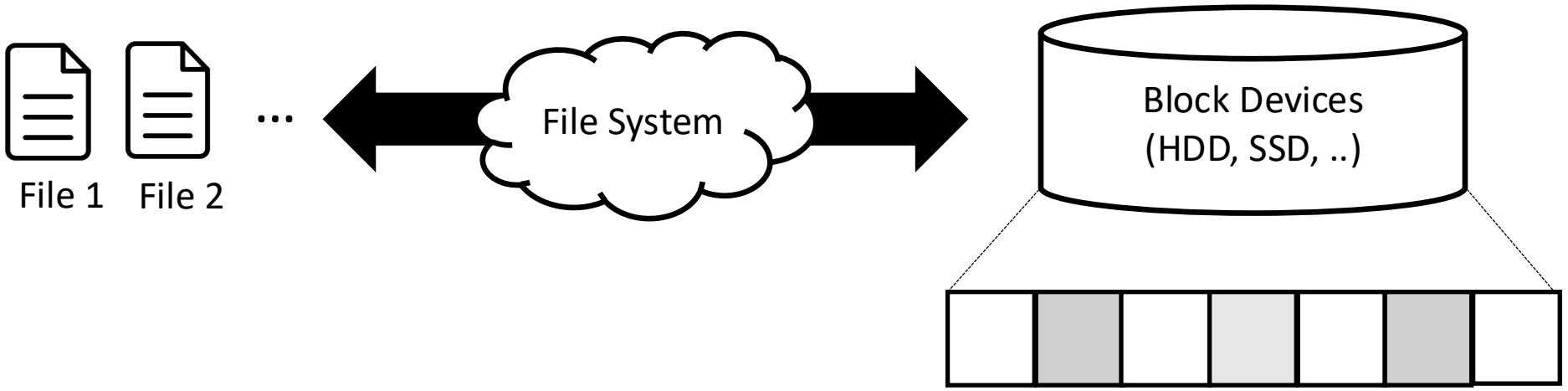
File System (1)

File system in xv6

File System

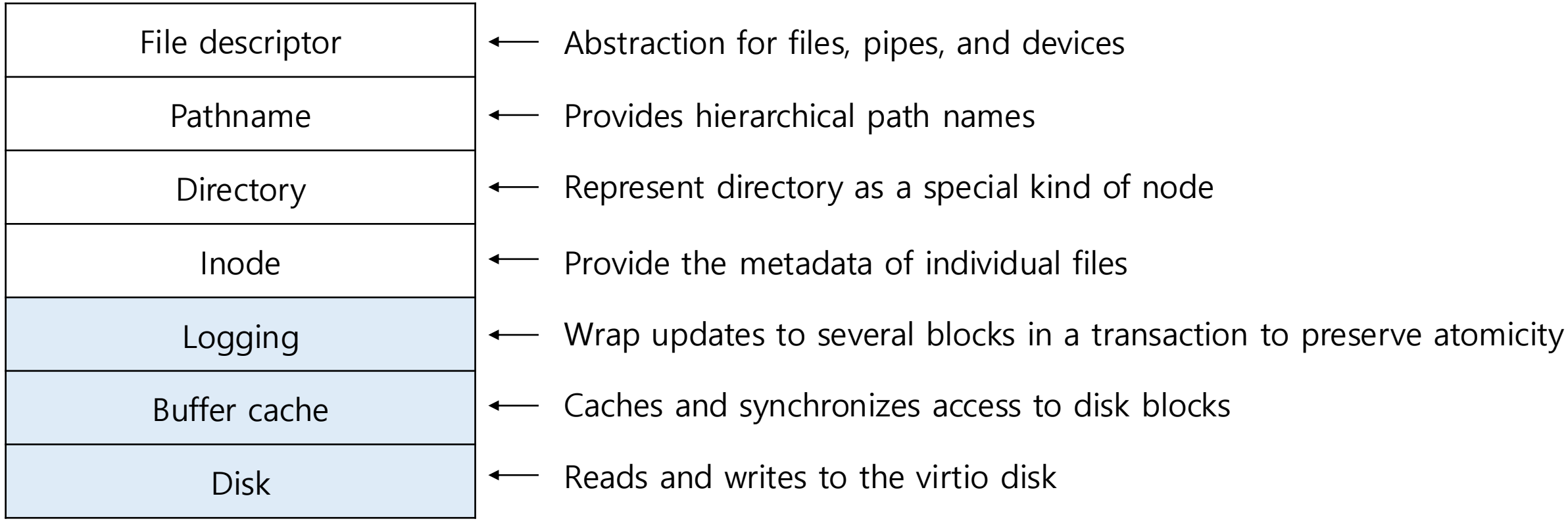
- System that organize and store data, supporting sharing of data among users and applications, as well as persistence so that data is still available after a reboot.



Challenges in File System

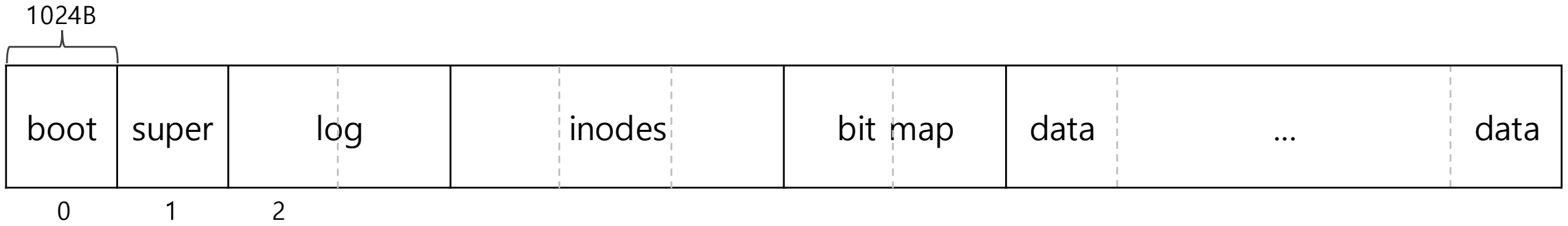
1. Need on-disk data structures.
   - To represent the tree of named directories and files
   - To record the identities of the blocks that hold each file's content
   - To record which areas of the disk are free.
2. Support crash recovery.
   - If a crash (ex. power failure) occurs, the file system must still work correctly after a restart
3. Consistency among multiple processes.
4. Maintain an in-memory cache for performance.

File System Layers



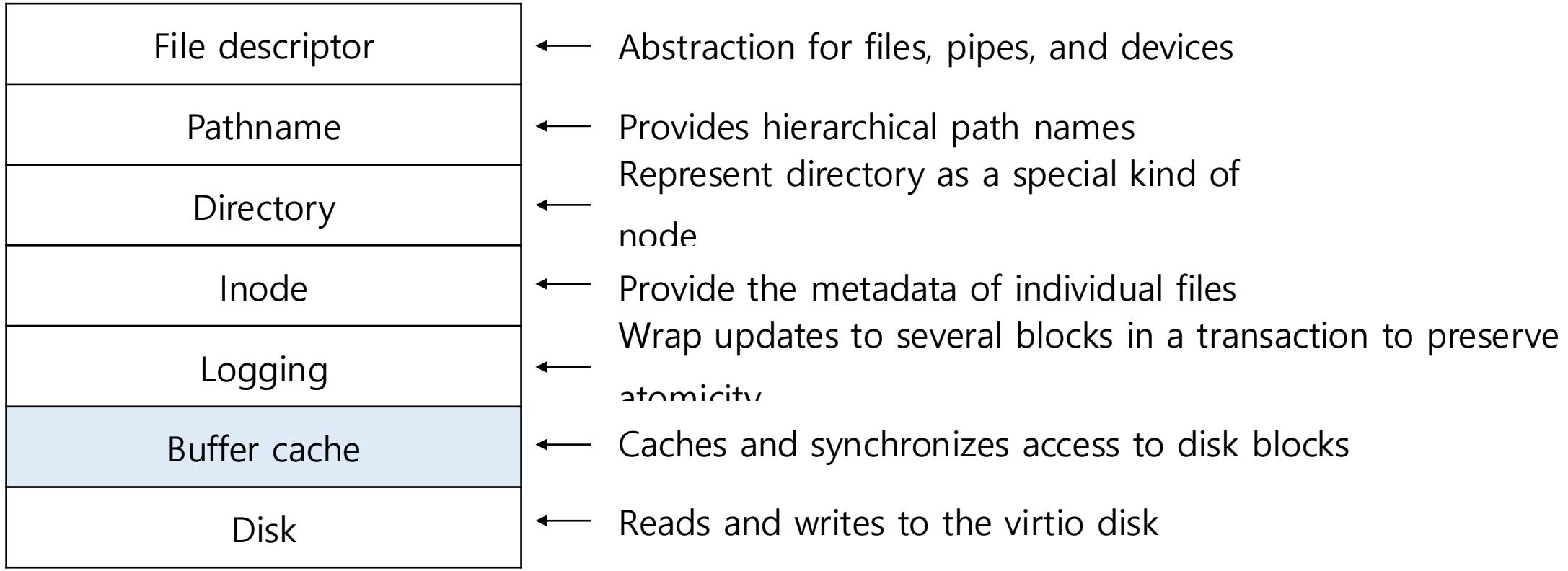
On-disk File System Structure

- xv6 divides the disk into several sections

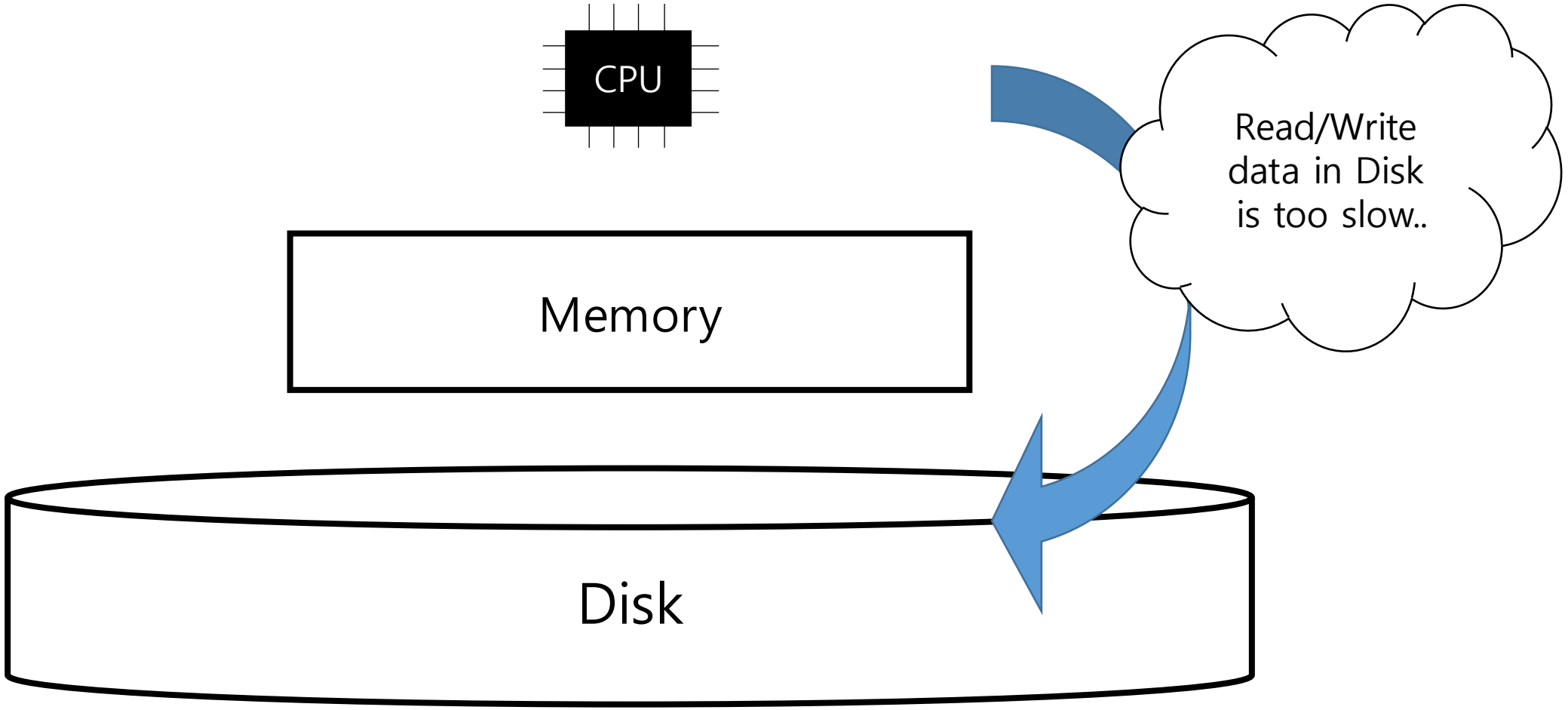


  - Boot (block 0): Uses only during boot.
  - Superblock (block 1): Contains metadata of the file system
  - Log (start from 2): Holds log
  - Inodes(after the log): Holds inodes
  - Bitmap(After the inodes): tracking which data blocks are in use
  - Data (remaining blocks): Holds real content

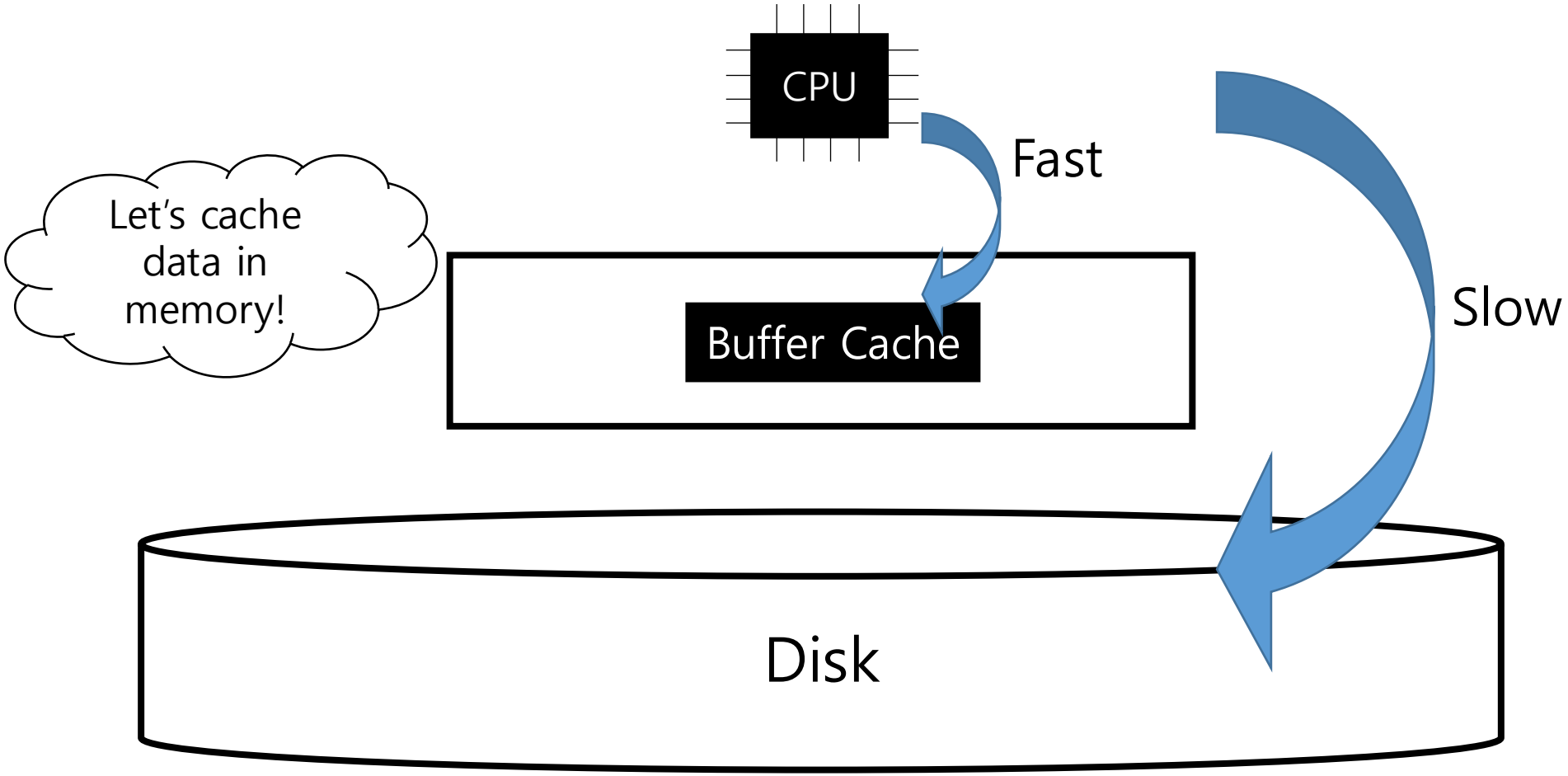
Buffer Cache Layer



Buffer cache



Buffer cache

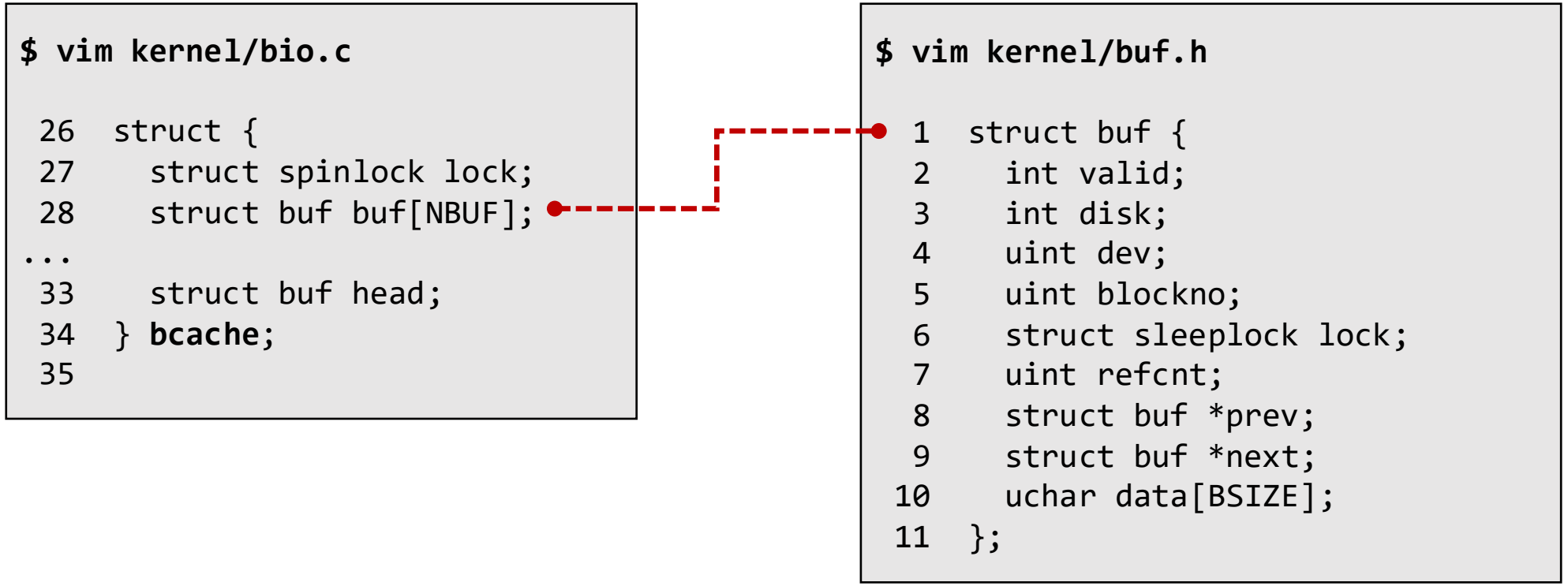


Buffer cache layer

- What is **buffer cache**? → A memory region that **temporarily holds disk blocks**
  - Acts as an intermediary between the file system and the physical disk.
  - Improve performance by avoiding frequent slow disk accesses.
  - Enforces synchronization when accessing shared disk data.
- Two goals of buffer cache:
  - **Caching** : Keeps frequently accessed blocks in memory to reduce disk access and improve performance.
  - **Synchronization** : Ensures that only one copy of disk block exists in memory and that only one kernel thread accesses it at a time.
- Interfaces: `bread, bwrite, bget, brelse`

Structure of buffer and buffer cache

- **`struct buf`** represents a single cached disk block in memory.
- **`struct bcache`** is the global structure that manages these buffers.



$ vim kernel/bio.c

```
 26  struct {
 27    struct spinlock lock;
 28    struct buf buf[NBUF];
...
 33    struct buf head;
 34  } bcache;
 35
```

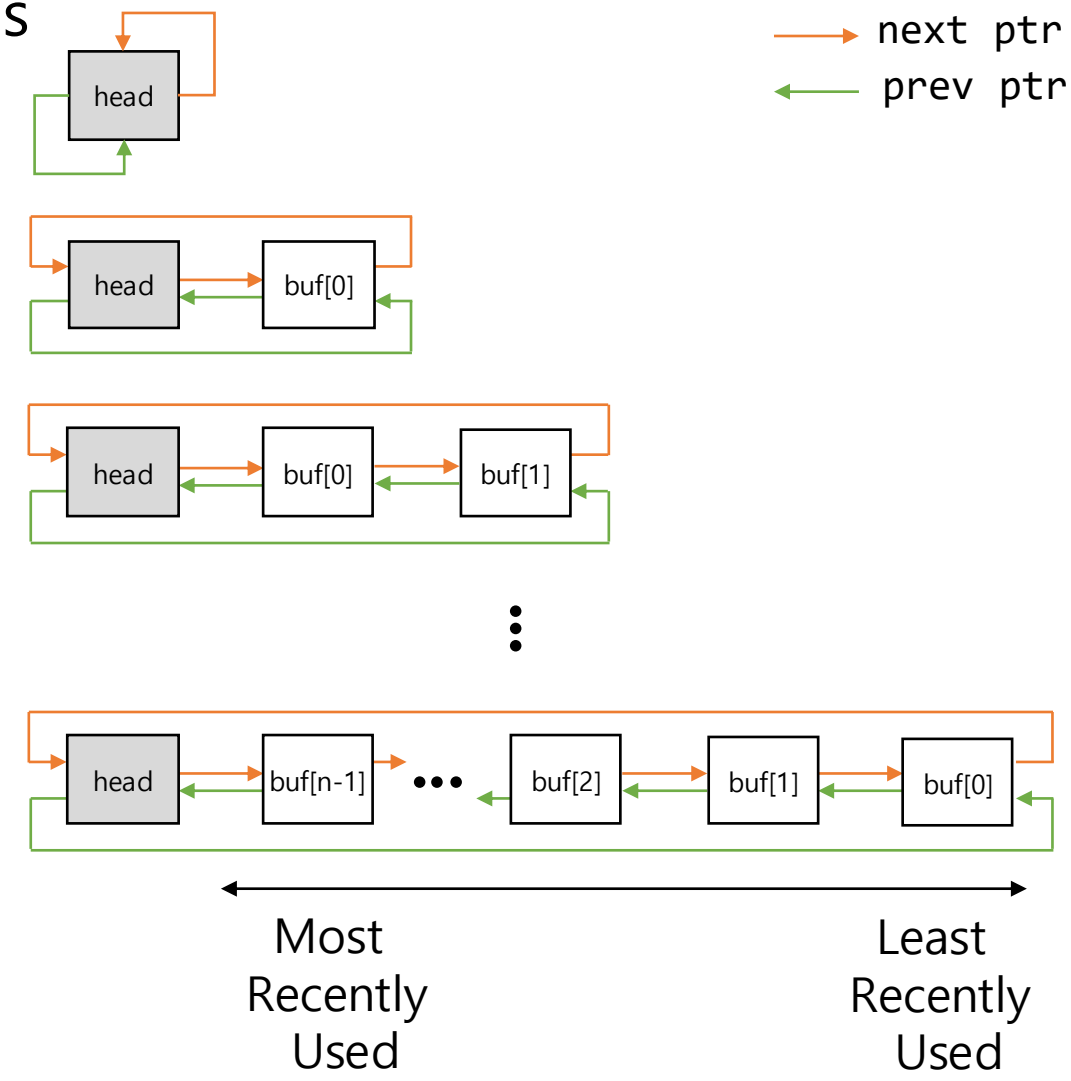
$ vim kernel/buf.h

```
  1  struct buf {
  2    int valid;
  3    int disk;
  4    uint dev;
  5    uint blockno;
  6    struct sleeplock lock;
  7    uint refcnt;
  8    struct buf *prev;
  9    struct buf *next;
 10    uchar data[BSIZE];
 11  };
```

binit

- Buffer cache is a doubly-linked list of buffers

```
$ vim kernel/bio.c

 36  void
 37  binit(void)
 38  {
 39    struct buf *b;
 40
 41    initlock(&bcache.lock, “bcache”);
 42
 43    // Create linked list of buffers
 44    bcache.head.prev = &bcache.head;
 45    bcache.head.next = &bcache.head;
 46    for(b = bcache.buf; b < bcache.buf+NBUF; b++){
 47      b->next = bcache.head.next;
 48      b->prev = &bcache.head;
 49      initsleeplock(&b->lock, “buffer”);
 50      bcache.head.next->prev = b;
 51      bcache.head.next = b;
 52    }
 53  }
```



bget

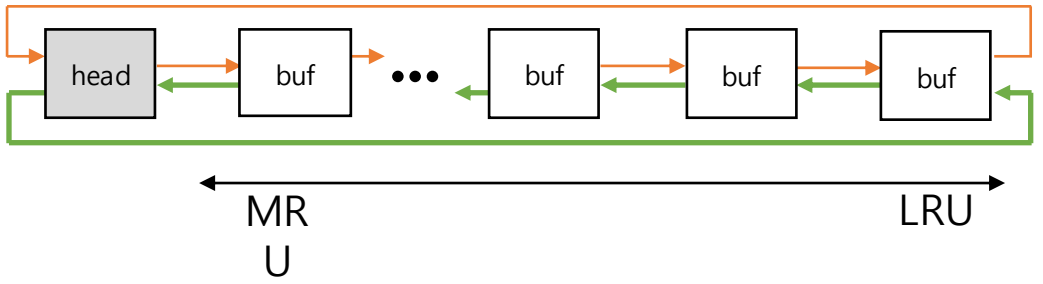
```
$ vim kernel/bio.c

 58  static struct buf*
 59  bget(uint dev, uint blockno)
 60  {
...
 66    for(b = bcache.head.next; b != &bcache.head; b = b->next){
 67      if(b->dev == dev && b->blockno == blockno){
 68        b->refcnt++;
 69        release(&bcache.lock);
 70        acquiresleep(&b->lock);
 71        return b;
 72      }
 73    }
...
 77    for(b = bcache.head.prev; b != &bcache.head; b = b->prev){
 78      if(b->refcnt == 0) {
 79        b->dev = dev;
 80        b->blockno = blockno;
 81        b->valid = 0;
 82        b->refcnt = 1;
 83        release(&bcache.lock);
 84        acquiresleep(&b->lock);
 85        return b;
 86      }
 87    }
 88    panic("bget: no buffers");
 89  }
```

- **bget( )** function checks if the requested (dev, blockno) block is already present in the buffer cache.
- If target buffer is in cache, just return the cached buffer

bget

```
$ vim kernel/bio.c

 58  static struct buf*
 59  bget(uint dev, uint blockno)
 60  {
...
 66    for(b = bcache.head.next; b != &bcache.head; b = b->next){
 67      if(b->dev == dev && b->blockno == blockno){
 68        b->refcnt++;
 69        release(&bcache.lock);
 70        acquiresleep(&b->lock);
 71        return b;
 72      }
 73    }
...
 77    for(b = bcache.head.prev; b != &bcache.head; b = b->prev){
 78      if(b->refcnt == 0) {
 79        b->dev = dev;
 80        b->blockno = blockno;
 81        b->valid = 0;
 82        b->refcnt = 1;
 83        release(&bcache.lock);
 84        acquiresleep(&b->lock);
 85        return b;
 86      }
 87    }
 88    panic("bget: no buffers");
 89  }
```

- **bget( )** function checks if the requested (dev, blockno) block is already present in the buffer cache.
- If target buffer is in cache, just return the cached buffer
- If not cached, reuses a free buffer based on the **LRU(Least Recently Used)** policy.



brelse

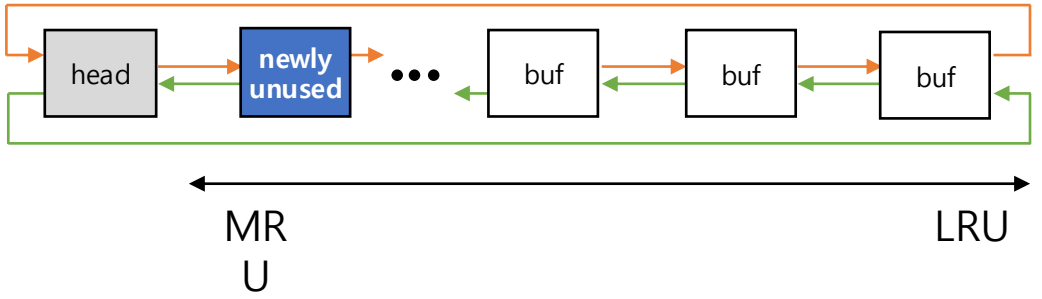
```
$ vim kernel/bio.c

116  void
117  brelse(struct buf *b)
118  {
119    if(!holdingsleep(&b->lock))
120     panic("brelse");
121
122    releasesleep(&b->lock);
123
124    acquire(&bcache.lock);
125    b->refcnt--;
126    if (b->refcnt == 0) {
127      // no one is waiting for it.
128      b->next->prev = b->prev;
129      b->prev->next = b->next;
130      b->next = bcache.head.next;
131      b->prev = &bcache.head;
132      bcache.head.next->prev = b;
133      bcache.head.next = b;
134    }
135
136    release(&bcache.lock);
137  }
```

- **`brelse()`** marks a buffer as available for reuse after use.
  - It releases the buffer's sleeplock, allowing other threads to acquire it.
  - It decrements the reference count
  - If it becomes 0, the buffer is no longer in use.

brelse

```
$ vim kernel/bio.c

116  void
117  brelse(struct buf *b)
118  {
119    if(!holdingsleep(&b->lock))
120     panic("brelse");
121
122    releasesleep(&b->lock);
123
124    acquire(&bcache.lock);
125    b->refcnt--;
126    if (b->refcnt == 0) {
127      // no one is waiting for it.
128      b->next->prev = b->prev;
129      b->prev->next = b->next;
130      b->next = bcache.head.next;
131      b->prev = &bcache.head;
132      bcache.head.next->prev = b;
133      bcache.head.next = b;
134    }
135
136    release(&bcache.lock);
137  }
```

- The unused buffer is moved to the **front** of the double-linked list, which represents the **most recently used**.
  - This reordering maintains the **LRU policy** : buffers used recently stay at the front, while older ones move to the back.



Buffer Cache Synchronization

```
$ vim kernel/bio.c

 58  static struct buf*
 59  bget(uint dev, uint blockno)
 60  {
...
 66    for(b = bcache.head.next; b != &bcache.head; b = b->next){
 67      if(b->dev == dev && b->blockno == blockno){
 68        b->refcnt++;
 69        release(&bcache.lock);
 70        acquiresleep(&b->lock);
 71        return b;
 72      }
 73    }
...
 77    for(b = bcache.head.prev; b != &bcache.head; b = b->prev){
 78      if(b->refcnt == 0) {
 79        b->dev = dev;
 80        b->blockno = blockno;
 81        b->valid = 0;
 82        b->refcnt = 1;
 83        release(&bcache.lock);
 84        acquiresleep(&b->lock);
 85        return b;
 86      }
 87    }
 88    panic("bget: no buffers");
 89  }
```

```
$ vim kernel/bio.c

116  void
117  brelse(struct buf *b)
118  {
119    if(!holdingsleep(&b->lock))
120     panic("brelse");
121
122    releasesleep(&b->lock);
123
124    acquire(&bcache.lock);
125    b->refcnt--;
126    if (b->refcnt == 0) {
127      // no one is waiting for it.
128      b->next->prev = b->prev;
129      b->prev->next = b->next;
130      b->next = bcache.head.next;
131      b->prev = &bcache.head;
132      bcache.head.next->prev = b;
133      bcache.head.next = b;
134    }
135
136    release(&bcache.lock);
137  }
```

- The returned buffer is protected by a sleeplock to ensure mutual exclusion.

Read and Write Buffers

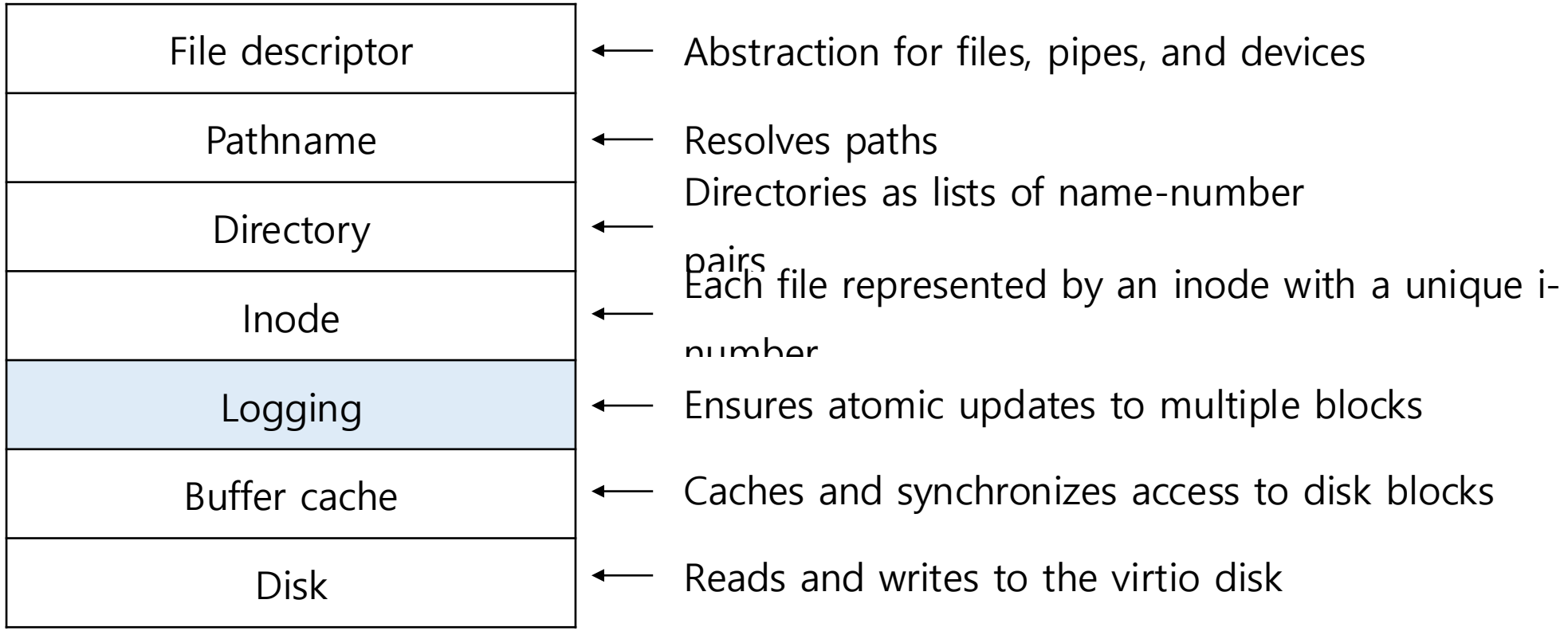
```
$ vim kernel/bio.c

 92  struct buf*
 93  bread(uint dev, uint blockno)
 94  {
 95    struct buf *b;
 96
 97    b = bget(dev, blockno);
 98    if(!b->valid) {
 99      virtio_disk_rw(b, 0);
100      b->valid = 1;
101    }
102    return b;
103  }
...
106  void
107  bwrite(struct buf *b)
108  {
109    if(!holdingsleep(&b->lock))
110      panic("bwrite");
111    virtio_disk_rw(b, 1);
112  }
```

- **`bread()`** loads a disk block into memory.
  - It obtains the requested buffer using **`bget()`**.
  - If **the buffer is not valid**, it reads the data from disk and sets **`valid`**.
- **`bwrite()`** writes a modified buffer back to disk.
  - It requires the buffer to be locked(sleeplock must be held)
  - It writes the buffer's contents to disk via

`void virtio_disk_rw(struct buf *b, int write)`

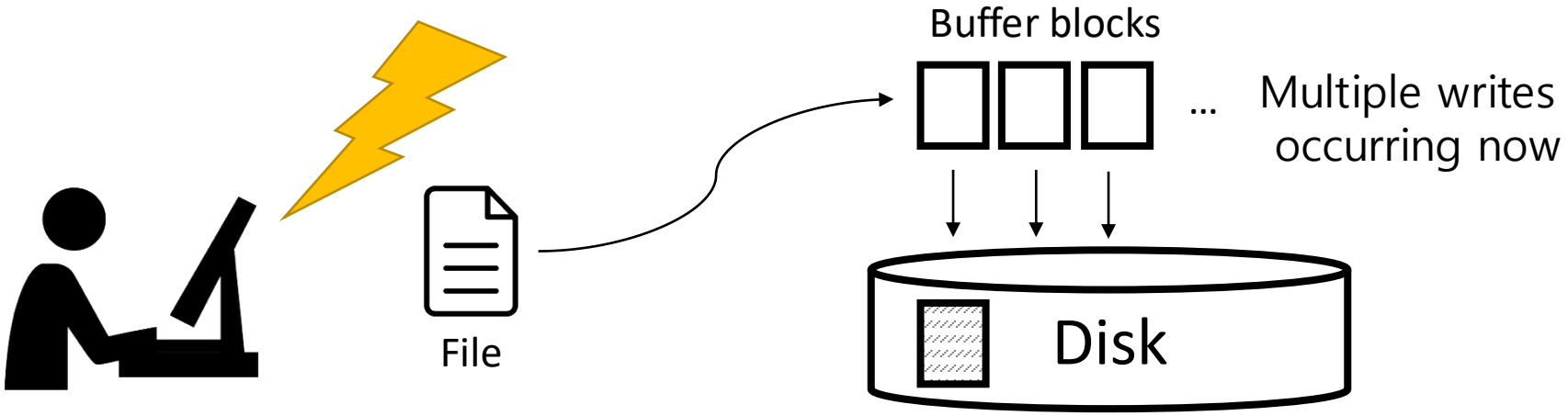
- **write to disk if `write==1`**
- **read from disk if `write==0`**

Logging layer



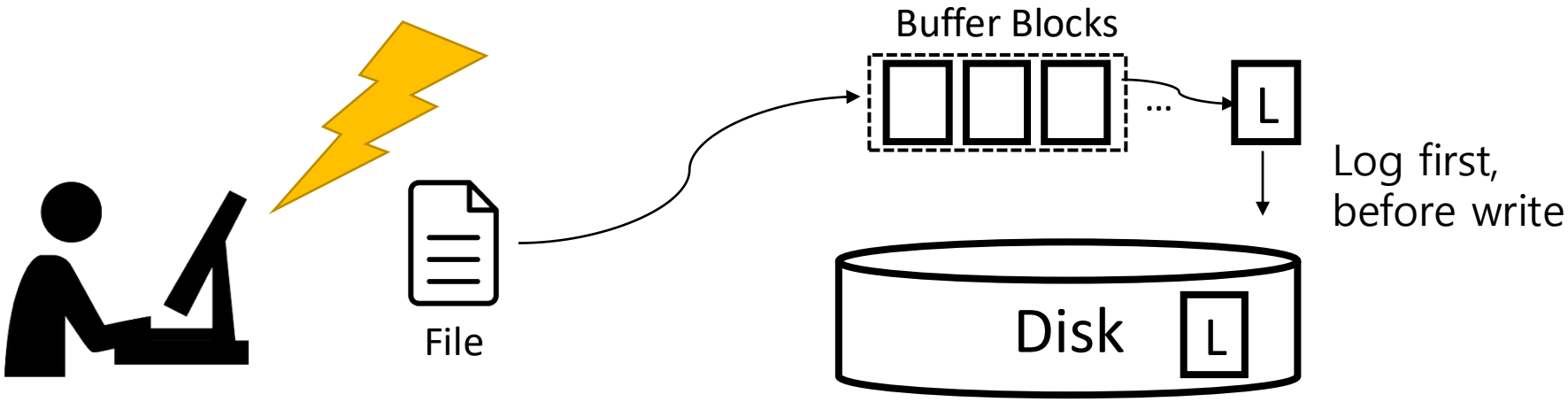
Logging

- Many file system operations involve multiple disk writes; so if a crash occurs during these operations, the disk may be left in an inconsistent state
  - ex) Inode referencing a freed block
  - This case can lead to data corruption or security issues, especially in multi-user environments.



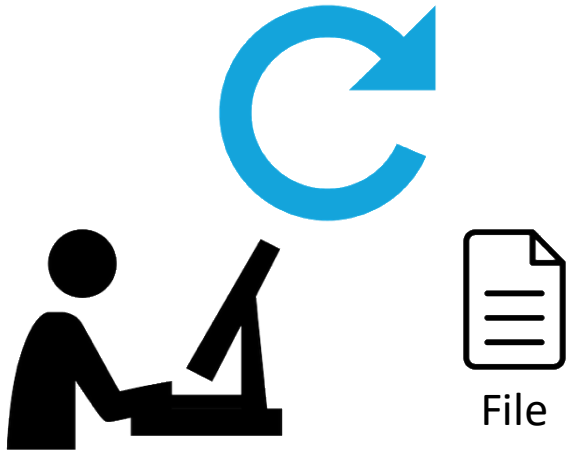
Logging

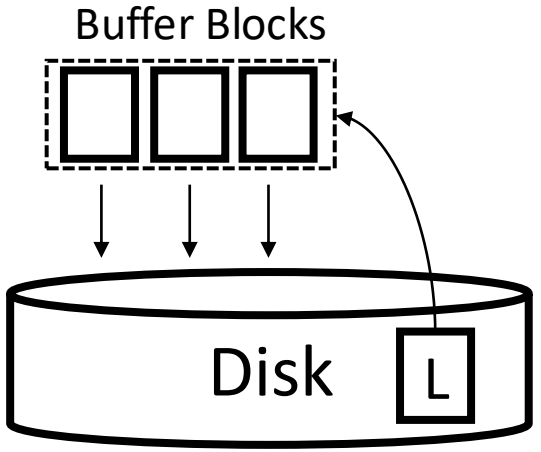
- Logging
  - Places a description of all the disk writes to log section on disk.
  - After the system has logged all of its writes, then commit.
  - Atomicity of writes are preserved based on that commit point.
    - Atomicity means "All or nothing".



Logging

- Recovery:
  - A process during reboot to mirror the written data to disk using logs, which is idempotent.





We can recover
by using written logs.

Logging Design in xv6

1. Log resides at a fixed location, specified in the superblock.
2. Each file system (FS) system call indicates the start and end of the sequence of writes: `begin_op()`, `end_op()`
3. Logging system accumulates the writes of multiple system calls into **one transaction** for efficiency and concurrency
4. Committing several transaction together, called group commit, allows the file system to batch several disk operations.
5. xv6 write system call breaks up large writes into multiple smaller writes that fit in the log.

Transaction

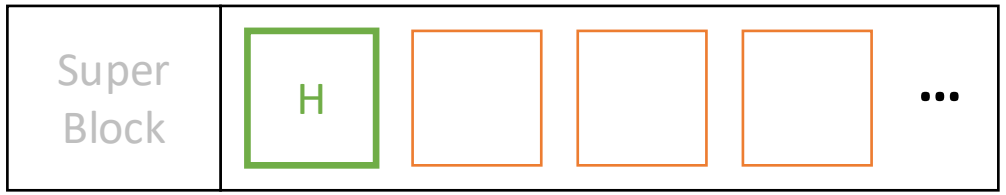
- Typical use of the log in a system call looks like this:

```
begin_op();           // Transaction Start
...
b = bread(..);        // Read Buffer
b->data[..] = ..;     // Modify Data
log_write(b);         // Log Write
...
end_op();             // Transaction End
```

Log structure

```
$ vim kernel/log.c

35  struct logheader {
36    int n;
37    int block[LOGSIZE];
38  };
39
40  struct log {
41    struct spinlock lock;
42    int start;
43    int size;
44    int outstanding;
45    int committing;
46    int dev;
47    struct logheader lh;
48  };
49  struct log log;
```

- **Header Block** stores number of modified blocks(n) and sector numbers of those blocks(`block[ ]`).
  - Sector numbers are written to log.start on disk.
- **Logged Blocks** contain actual modified block copies.
  - These are written to the log area on disk before being applied to the file system.



Log space

begin_op

```
$ vim kernel/log.c

127  void
128  begin_op(void)
129  {
130    acquire(&log.lock);
131    while(1){
132      if(log.committing){
133        sleep(&log, &log.lock);
134      } else if(log.lh.n + (log.outstanding+1)*MAXOPBLOCKS > LOGSIZE){
135        // this op might exhaust log space; wait for commit.
136        sleep(&log, &log.lock);
137      } else {
138        log.outstanding += 1;
139        release(&log.lock);
140        break;
141      }
142    }
143  }
```

Wait if someone is committing in the logging system now.

Wait if the free log space is short because of handling executing FS system calls.

- **begin_op( )** is called at the start of a new file system operation
- Determines whether to proceed logging or not

begin_op

```
$ vim kernel/log.c

127  void
128  begin_op(void)
129  {
130    acquire(&log.lock);
131    while(1){
132      if(log.committing){
133        sleep(&log, &log.lock);
134      } else if(log.lh.n + (log.outstanding+1)*MAXOPBLOCKS > LOGSIZE){
135        // this op might exhaust log space; wait for commit.
136        sleep(&log, &log.lock);
137      } else {
138        log.outstanding += 1;
139        release(&log.lock);
140        break;
141      }
142    }
143  }
```

log.outstanding increment means:

1. Log space is reserved (by line 134)
2. Group commit (we will see soon)

- **begin_op( )** is called at the start of a new file system operation
- Determines whether to proceed logging or not

log_write

```
$ vim kernel/log.c

215  void
216  log_write(struct buf *b)
217  {
218    int i;
219
220    acquire(&log.lock);
221    if (log.lh.n >= LOGSIZE || log.lh.n >= log.size - 1)
222      panic("too big a transaction");
223    if (log.outstanding < 1)
224      panic("log_write outside of trans");
225
226    for (i = 0; i < log.lh.n; i++) {
227      if (log.lh.block[i] == b->blockno)
228        break;
229    }
230    log.lh.block[i] = b->blockno;
231    if (i == log.lh.n) {  // Add new block to log?
232      bpin(b);
233      log.lh.n++;
234    }
235    release(&log.lock);
236  }
```

- `log_write` records the block number in memory to reserve a slot in the log on disk.

Log absorption:
Allocates the same slot in the log when the block is written multiple times.

Prevent eviction:
Pins the buffer in the block cache to prevent the block cache from evicting it

end_op

```
$ vim kernel/log.c

147  void
148  end_op(void)
149  {
150    int do_commit = 0;
151
152    acquire(&log.lock);
153    log.outstanding -= 1;
154    if(log.committing)
155      panic("log.committing");
156    if(log.outstanding == 0){
157      do_commit = 1;
158      log.committing = 1;
159    } else {
...
163      wakeup(&log);
164    }
165    release(&log.lock);
...
```

Group commit:
Commit when there is **no outstanding** FS system calls.

- **end_op( )** is called at the end of a file system operation.
- Decrements the count of outstanding system calls. If the count is zero, it commits the current transaction.

commit

```
$ vim kernel/log.c

147  void
148  end_op(void)
149  {
150    int do_commit = 0;
...
156    if(log.outstanding == 0){
157      do_commit = 1;
158      log.committing = 1;
159    } else {
...
167    if(do_commit){
168      // call commit w/o holding locks, since not allowed
169      // to sleep with locks.
170      commit();
171      acquire(&log.lock);
172      log.committing = 0;
173      wakeup(&log);
174      release(&log.lock);
175    }
176  }
```

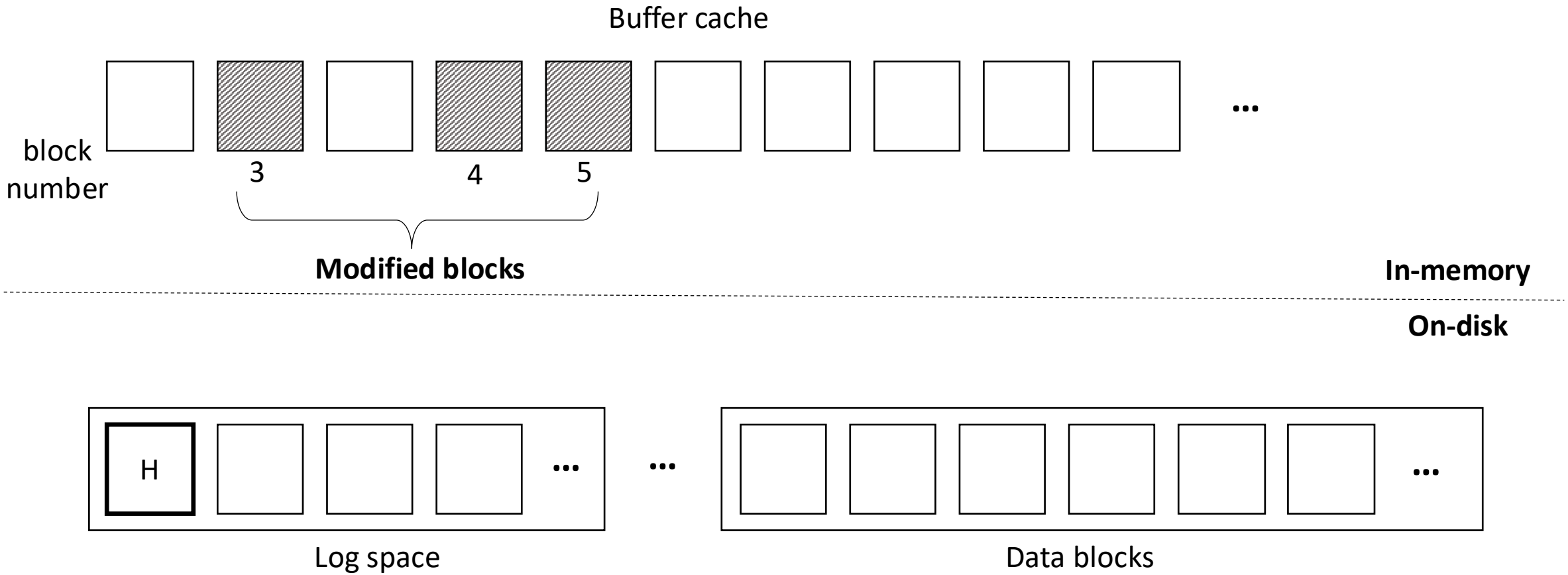
- There are 4 stages in commit.

```
$ vim kernel/log.c

194  static void
195  commit()
196  {
197    if (log.lh.n > 0) {
198      write_log();
199      write_head();
200      install_trans(0);
201      log.lh.n = 0;
202      write_head();
203    }
204  }
```

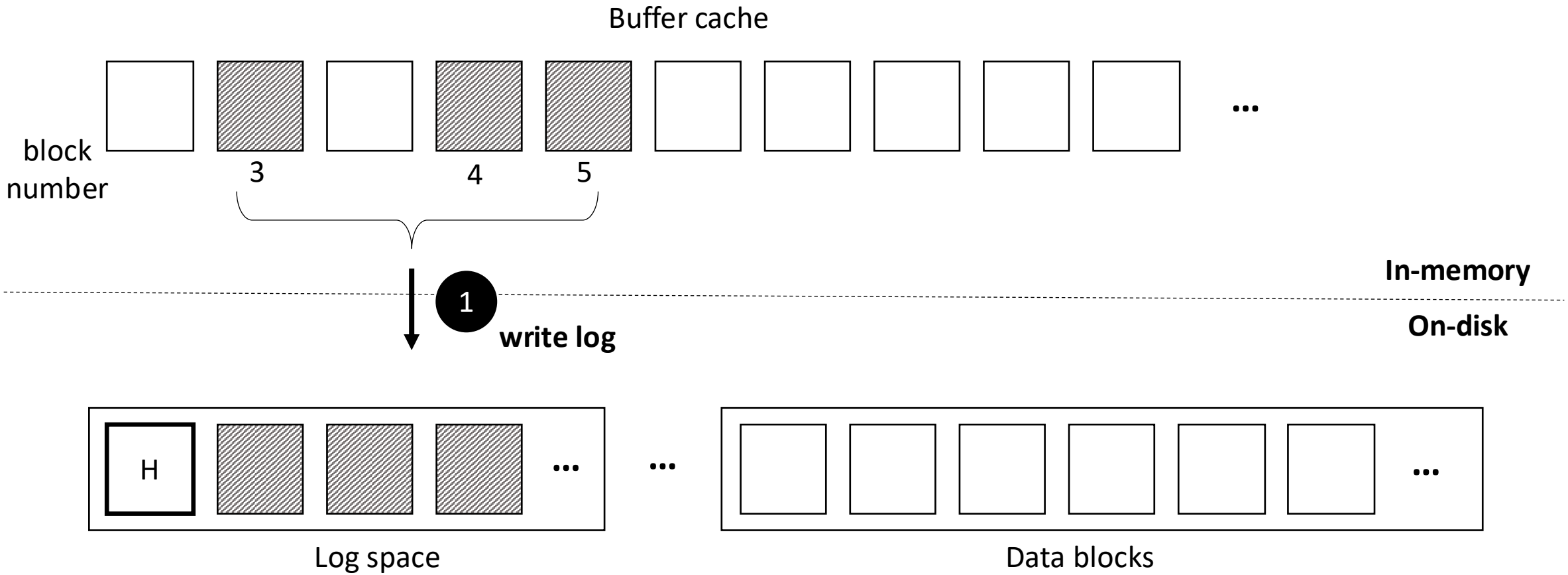
Commit steps:

1. Write modified blocks from cache to log.
2. Write header to disk.
3. Install writes to home locations.
4. Erase the transaction from the log.

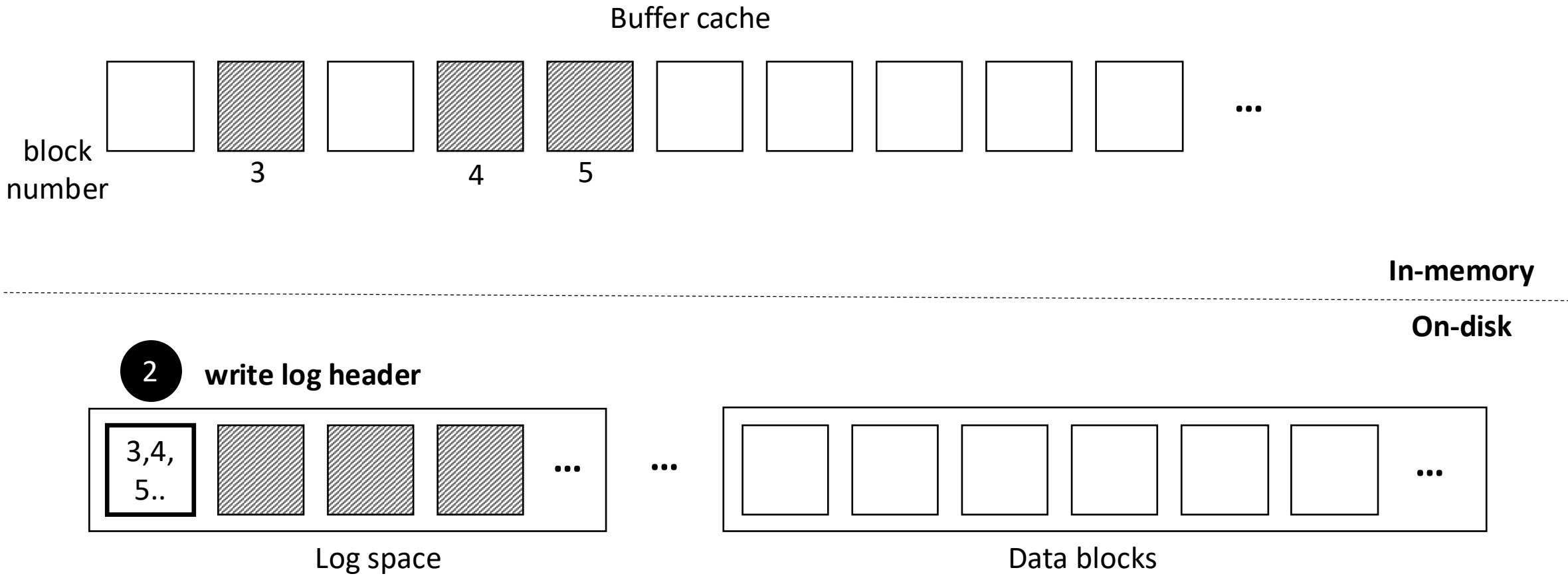
commit


Buffer cache
block number
3
4
5
...
Modified blocks
In-memory
On-disk
H
...
...
...
Log space
Data blocks

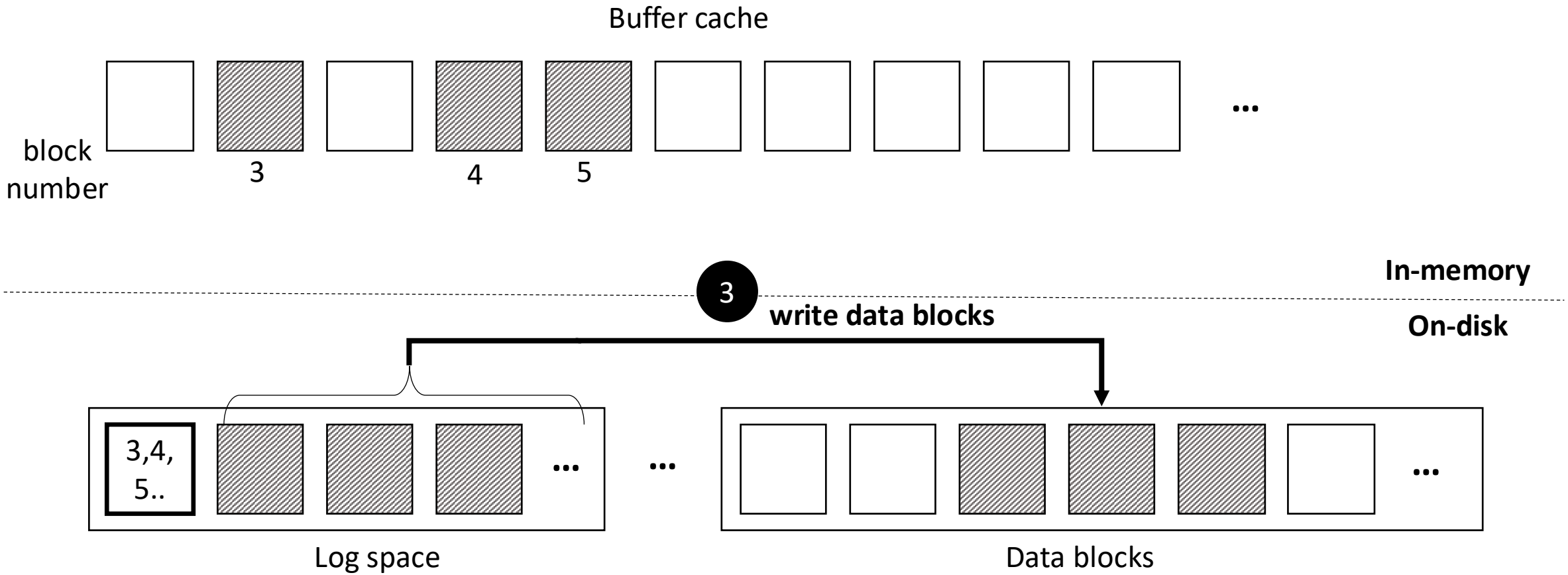
commit


Buffer cache
block number
3
4
5
...
In-memory
On-disk
1
write log
H
...
...
...
Log space
Data blocks

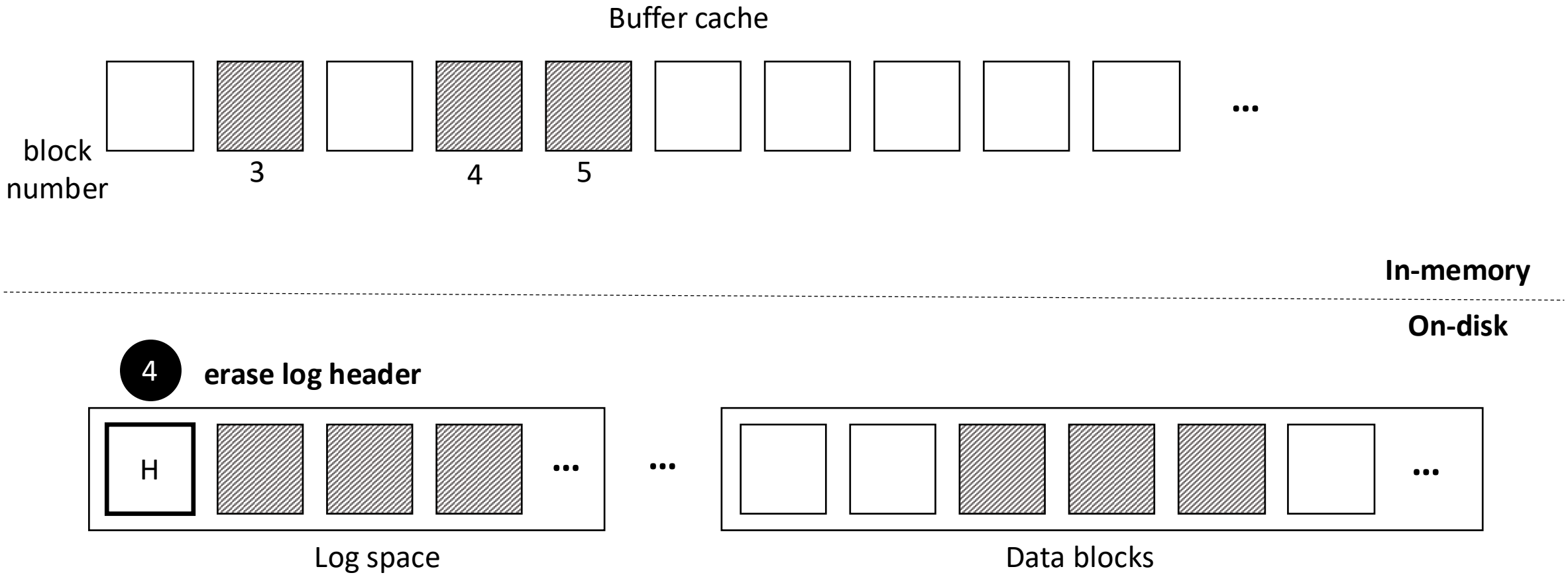
commit


Buffer cache
block number
3
4
5
...
In-memory
On-disk
2
write log header
3,4,
5..
...
...
...
Log space
Data blocks

commit

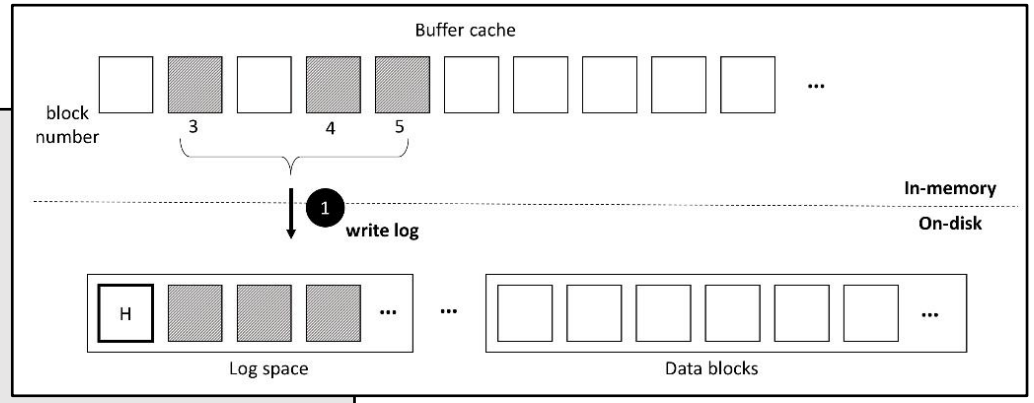

Buffer cache
block number
3
4
5
...
In-memory
On-disk
3
write data blocks
3,4,
5..
...
...
...
Log space
Data blocks

commit


Buffer cache
block number
3
4
5
...
In-memory
On-disk
4
erase log header
H
...
...
...
Log space
Data blocks

commit-(1): write_log

- `write_log` copies each block modified in the transaction from the buffer cache to its slot in the log on disk.

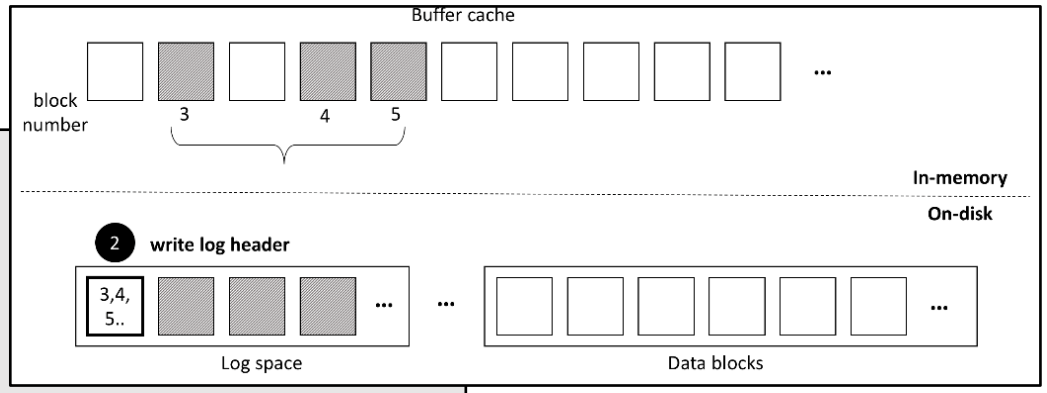



```
$ vim kernel/log.c

179 static void
180 write_log(void)
181 {
182   int tail;
183
184   for (tail = 0; tail < log.lh.n; tail++) {
185     struct buf *to = bread(log.dev, log.start+tail+1); // log block
186     struct buf *from = bread(log.dev, log.lh.block[tail]); // cache block
187     memmove(to->data, from->data, BSIZE);
188     bwrite(to);  // write the log
189     brelse(from);
190     brelse(to);
191   }
192 }
```

Copy blocks from buffer cache to log on disk.

commit-(2): write_head

- `write_head` writes the header block to disk.

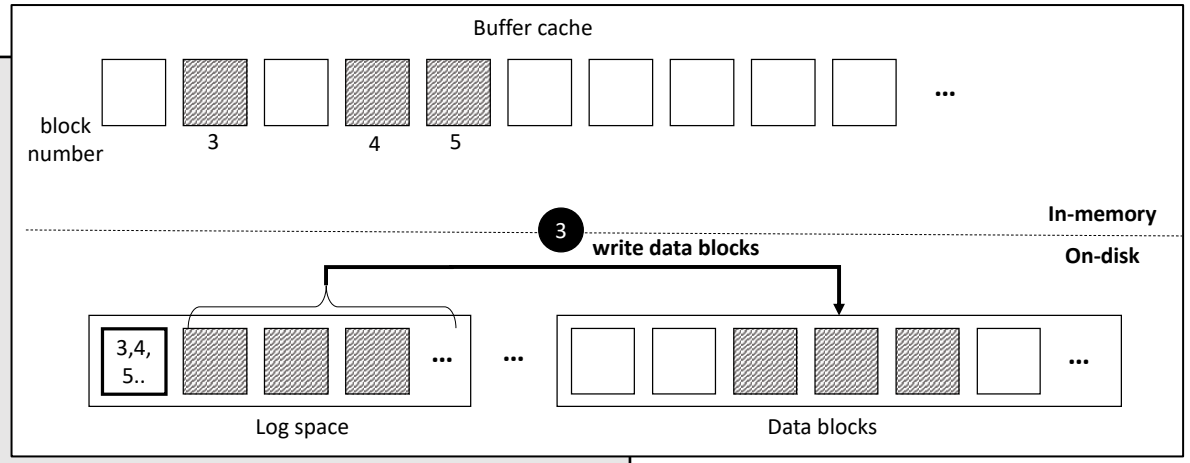



```
$ vim kernel/log.c

103 static void
104 write_head(void)
105 {
106   struct buf *buf = bread(log.dev, log.start);
107   struct logheader *hb = (struct logheader *) (buf->data);
108   int i;
109   hb->n = log.lh.n;
110   for (i = 0; i < log.lh.n; i++) {
111     hb->block[i] = log.lh.block[i];
112   }
113   bwrite(buf);
114   brelse(buf);
115 }
```

Actual Commit Point:
After written log header on disk, recovery will restore logged block by reading redo logs.

commit-(3): install_trans

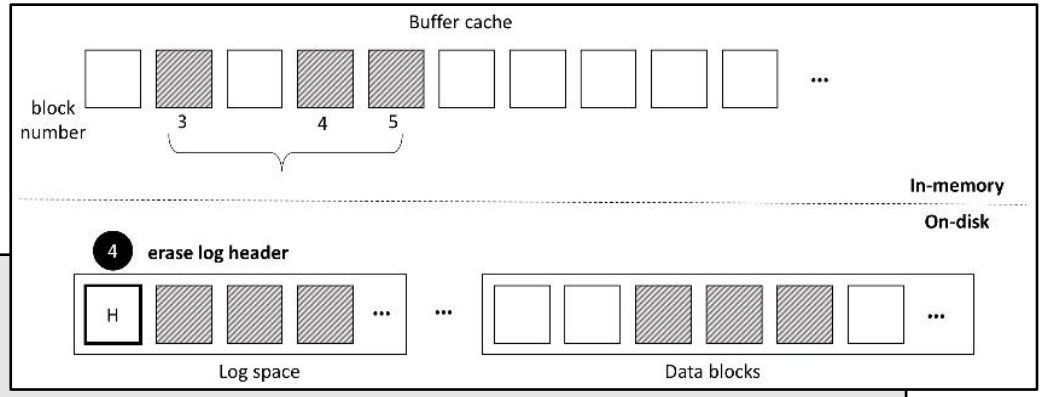
- `install_trans` reads each block from the log and writes it to the proper place in the file system.




```
$ vim kernel/log.c

 66 static void
 67 install_trans(int recovering)
 68 {
 69   int tail;
 70
 71   for (tail = 0; tail < log.lh.n; tail++) {
 72     if(recovering) {
 73       printf("recovering tail %d dst %d\n", tail, log.lh.block[tail]);
 74     }
 75     struct buf *lbuf = bread(log.dev, log.start+tail+1); // read log block
 76     struct buf *dbuf = bread(log.dev, log.lh.block[tail]); // read dst
 77     memmove(dbuf->data, lbuf->data, BSIZE);  // copy block to dst
 78     bwrite(dbuf);  // write dst to disk
 79     if(recovering == 0)
 80       bunpin(dbuf)
 81     brelse(lbuf);
 82     brelse(dbuf);
 83   }
 84 }
```

commit-(4): Erase log

- Writes the log header with a count of zero: must set 0 before next transaction starts writing logged blocks.




```
$ vim kernel/log.c

194 static void
195 commit()
196 {
197   if (log.lh.n > 0) {
198     write_log();
199     write_head();
200     install_trans(0);
201     log.lh.n = 0;
202     write_head();
203   }
204 }
```

```
$ vim kernel/log.c

103 static void
104 write_head(void)
105 {
106   struct buf *buf = bread(log.dev, log.start);
107   struct logheader *hb = (struct logheader *) (buf->data);
108   int i;
109   hb->n = log.lh.n;
110   for (i = 0; i < log.lh.n; i++) {
111     hb->block[i] = log.lh.block[i];
112   }
113   bwrite(buf);
114   brelse(buf);
115 }
```

Recovery

- `recover_from_log` reads the log header and mimics the actions of end_op.

```
$ vim kernel/log.c

117 static void
118 recover_from_log(void)
119 {
120   read_head();
121   install_trans(1);
122   log.lh.n = 0;
123   write_head();
124 }
```

```
$ vim kernel/log.c

 87 static void
 88 read_head(void)
 89 {
 90   struct buf *buf = bread(log.dev, log.start);
 91   struct logheader *lh = (struct logheader *) (buf->data);
 92   int i;
 93   log.lh.n = lh->n;
 94   for (i = 0; i < log.lh.n; i++) {
 95     log.lh.block[i] = lh->block[i];
 96   }
 97   brelse(buf);
 98 }
```

Filewrite System call

```
$ vim kernel/file.c

134  int
135  filewrite(struct file *f, char *addr, int n)
136  {
...
155      while(i < n){
...
160        begin_op();
161        ilock(f->ip);
162        if ((r = writei(f->ip, addr + i, f->off, n1)) > 0)
163          f->off += r;
164        iunlock(f->ip);
165        end_op();
...
171        i += r;
172      }
...
179  }
```

- Loop breaks up large writes into individual transactions to avoid overflowing the log.

Thank you